

Week-4: DAY-18 Hands-On Keerthivasan R

Level-1 Problem 1: Student Grade Calculator

Scenario:

A school wants to calculate the average marks of a student using a class-based approach.

Requirements:

1. Create a class Student.
2. Create method CalculateAverage(int m1, int m2, int m3).
3. Return the average marks.
4. Display grade based on average.

Technical Constraints:

1. Use return type double for average.
2. Avoid hard-coded values.

Expectations:

Clear separation of logic inside methods.

Learning Outcome:

Learn method creation, return values, and basic OOP concepts.

Sample Input:

80 70 90

Sample Output:

Average = 80, Grade = A

CODE:

```
using System;

namespace HandsOnDAY18
{
    class Student
    {
        public double CalculateAverage(int m1, int m2, int m3)
        {
            return (m1 + m2 + m3) / 3.0;
        }

        public string CalculateGrade(double average)
```

```

    {
        if (average >= 80)
            return "A";
        else if (average >= 65)
            return "B";
        else if (average >= 50)
            return "C";
        else if (average >= 35)
            return "D";
        else
            return "F";
    }
}
class StudentGradeCalculator
{
    static void Main(string[] args)
    {
        Student student = new Student();

        Console.Write("Enter the Mark1: ");
        int m1 = Convert.ToInt32(Console.ReadLine());

        Console.Write("Enter the Mark2: ");
        int m2 = Convert.ToInt32(Console.ReadLine());

        Console.Write("Enter the Mark3: ");
        int m3 = Convert.ToInt32(Console.ReadLine());

        double average = student.CalculateAverage(m1, m2, m3);

        string grade = student.CalculateGrade(average);

        Console.WriteLine("Average = " + average + ", Grade = " + grade);
    }
}

```

OUTPUT:

```

Enter the Mark1: 50
Enter the Mark2: 60
Enter the Mark3: 90
Average = 66.6666666666667, Grade = B

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-17\HandsOnDAY18
\HandsOnDAY18\bin\Debug\HandsOnDAY18.exe (process 10784) exited with code 0 (0x0).
Press any key to close this window . . .|

```

Level-2 Problem 1: Bank Account with Encapsulation

Scenario:

A bank wants to manage customer accounts securely using encapsulation.

Requirements:

1. Create class BankAccount.
2. Private field: balance.
3. Public methods: Deposit(double amount), Withdraw(double amount).
4. Method GetBalance() to return balance.
5. Prevent withdrawal if insufficient balance.

Technical Constraints:

1. Balance must be private.
2. Access balance only through public methods.
3. Use appropriate return types.

Expectations:

Proper use of encapsulation and object-oriented principles.

Learning Outcome:

Understand encapsulation, access modifiers, and secure data handling.

Sample Input:

Deposit 1000, Withdraw 300

Sample Output:

Current Balance = 700

CODE:

```
using System;

namespace HandsOnDAY18
{
    class BankAccount
    {
        private double balance;

        // Method to deposit money
        public void Deposit(double amount)
        {
```

```

        balance += amount;
    }
    // Method to withdraw money
    public void Withdraw(double amount)
    {
        if (amount > balance)
        {
            Console.WriteLine("Insufficient Balance");
        }
        else
        {
            balance -= amount;
        }
    }
    // Method to get balance
    public double GetBalance()
    {
        return balance;
    }
}
class Bank_Account_with_Encapsulation {
    static void Main(string[] args) {
        BankAccount account = new BankAccount();
        // Sample Input
        Console.Write("Enter the amt to deposit: ");
        double deposit = Convert.ToDouble(Console.ReadLine());
        account.Deposit(deposit);
        Console.Write("Enter the amt to Withdraw: ");
        double withdraw = Convert.ToDouble(Console.ReadLine());
        account.Withdraw(withdraw);
        // Output
        Console.WriteLine("Current Balance = " + account.GetBalance());
    }
}

```

OUTPUT:

```

Enter the amt to deposit: 20000
Enter the amt to Withdraw: 1000
Current Balance = 19000

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-17\HandsOnDAY18
\HandsOnDAY18\bin\Debug\HandsOnDAY18.exe (process 21956) exi
ted with code 0 (0x0).
Press any key to close this window . . .|

```

Level-2 Problem 2: Bank Account Management System

Scenario:

A bank wants to develop a simple console-based application to manage customer bank accounts. The system should protect account balance information and allow controlled access using properties.

Requirements:

1. Create a BankAccount class with private fields for account number and balance.
2. Use properties to allow controlled access to account number and balance.
3. Implement Deposit and Withdraw methods with proper validation.
4. Prevent withdrawal if balance is insufficient.

Technical Constraints:

- Use private fields with public properties.
- Apply encapsulation and data hiding.
- No direct access to balance field from outside the class.

Expectations:

- Demonstrate correct use of access modifiers.
- Validate negative deposit or withdrawal amounts.
- Display updated balance after each transaction.

Learning Outcome:

- Understand encapsulation using properties.
- Apply data hiding effectively.
- Implement validation logic inside class methods.

Sample Input:

Deposit = 5000, Withdraw = 2000

Sample Output:

Current Balance = 3000

CODE:

```
using System;
```

```

namespace HandsOnDAY18 {
    class BankAcc
    {
        // Private fields
        private string accountNumber;
        private double balance;

        // Property for Account Number
        public string AccountNumber
        {
            get { return accountNumber; }
            set { accountNumber = value; }
        }

        // Property for Balance (Read Only outside)
        public double Balance
        {
            get { return balance; }
            private set { balance = value; }
        }

        // Deposit Method
        public void Deposit(double amount)
        {
            if (amount <= 0)
            {
                Console.WriteLine("Invalid deposit amount");
                return;
            }

            Balance += amount;
            Console.WriteLine("Amount Deposited: " + amount);
            Console.WriteLine("Current Balance = " + Balance);
        }

        // Withdraw Method
        public void Withdraw(double amount)
        {
            if (amount <= 0)
            {
                Console.WriteLine("Invalid withdrawal amount");
                return;
            }

            if (amount > Balance)
            {
                Console.WriteLine("Insufficient Balance");
                return;
            }

            Balance -= amount;
            Console.WriteLine("Amount Withdrawn: " + amount);
            Console.WriteLine("Current Balance = " + Balance);
        }
    }

    class Bank_Account_Management_System
    {
        static void Main()
        {
            BankAcc acc = new BankAcc();

            acc.AccountNumber = "ACC1001";
        }
    }
}

```

```

        acc.Deposit(5000);
        acc.Withdraw(2000);

        Console.WriteLine("Final Balance = " + acc.Balance);
    }
}

```

OUTPUT:

```

Amount Deposited: 5000
Current Balance = 5000
Amount Withdrawn: 2000
Current Balance = 3000
Final Balance = 3000

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-17\HandsOnDAY18
\HandsOnDAY18\bin\Debug\HandsOnDAY18.exe (process 29660) exi
ted with code 0 (0x0).
Press any key to close this window . . .|

```

Level-2 Problem 3: Employee Salary Calculator

Scenario:

A company wants to calculate employee salaries using inheritance. All employees have a base salary, but different roles calculate bonuses differently.

Requirements:

1. Create a base class Employee with Name and BaseSalary properties.
2. Create derived classes Manager and Developer.
3. Override a virtual method CalculateSalary().

4. Manager gets 20% bonus, Developer gets 10% bonus.

Technical Constraints:

- Use inheritance and method overriding.
- Apply polymorphism using base class reference.
- Use properties for salary details.

Expectations:

- Demonstrate runtime polymorphism.
- Avoid code duplication.
- Display final calculated salary.

Learning Outcome:

- Understand inheritance hierarchy.
- Implement polymorphism using virtual and override.
- Write reusable and extensible code.

Sample Input:

BaseSalary = 50000

Sample Output:

Manager Salary = 60000, Developer Salary = 55000

CODE:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace HandsOnDAY18
{
    class Employee {
        // properties
        public string name { get; set; }
        public double baseSalary { get; set; }
        //virtual method
        public virtual double CalculateSalary() {
            return baseSalary;
        }
    }
    class Manager : Employee {
        //override method
        public override double CalculateSalary() {
            return baseSalary + (baseSalary * 0.20);
        }
    }
}
```



```

    }
}
class Developer : Employee {
    //override method
    public override double CalculateSalary()
    {
        return baseSalary + (baseSalary * 0.10);
    }
}
internal class Employee_Salary_Calcullator
{
    static void Main(string[] args) {

        Console.Write("Enter the base salary: ");
        double baseSalary = Convert.ToDouble(Console.ReadLine());

        //polymorphism using base class reference
        Employee manager = new Manager();
        manager.baseSalary = baseSalary;

        Employee developer = new Developer();
        developer.baseSalary = baseSalary;

        Console.WriteLine("Manager Salary: " + manager.CalculateSalary());
        Console.WriteLine("Developer Salary: " + developer.CalculateSalary());

    }
}
}

```

OUTPUT:

```

Enter the base salary: 20000
Manager Salary: 24000
Developer Salary: 22000

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-17\HandsOnDAY18
\HandsOnDAY18\bin\Debug\HandsOnDAY18.exe (process 24096) exit
ed with code 0 (0x0).
Press any key to close this window . . .|

```

Level-2 Problem 4: Online Shopping Cart System

Scenario:

An e-commerce platform needs a flexible cart system where different product types calculate discounts differently.

Requirements:

1. Create a base class Product with properties Name and Price.
2. Create derived classes Electronics and Clothing.
3. Implement a virtual method CalculateDiscount().
4. Electronics get 5% discount, Clothing gets 15% discount.
5. Use encapsulation to protect price updates.

Technical Constraints:

- Use private fields with public properties.
- Apply inheritance and method overriding.
- Prevent negative price assignment.

Expectations:

- Demonstrate polymorphic behavior in cart processing.
- Apply data validation inside properties.
- Calculate and display final price after discount.

Learning Outcome:

- Combine encapsulation and polymorphism.
- Design extensible product hierarchy.
- Implement business logic in overridden methods.

Sample Input: Electronics Price = 20000

Sample Output: Final Price after 5% discount = 19000

CODE:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Runtime.Remoting.Messaging;
using System.Text;
```

```

using System.Threading.Tasks;

namespace HandsOnDAY18
{
    class Product {
        //private fields
        private string name;
        private double price;

        //property for name
        public string Name {
            get { return name; }
            set { name = value; }
        }
        //property for price
        public double Price {
            get { return price; }
            set {
                if (value < 0)
                {
                    Console.WriteLine("Price cannot be negative");
                }
                else {
                    price = value;
                }
            }
        }
        public virtual double CalculateDiscount() {
            return price;
        }
    }
    class Electronics : Product
    {
        public override double CalculateDiscount()
        {
            return Price - (Price * 0.05);
        }
    }

    class Clothing : Product
    {
        public override double CalculateDiscount()
        {
            return Price - (Price * 0.15);
        }
    }
    internal class Online_Shopping_Cart_System
    {
        static void Main(string[] args) {

            //polymorphism
            Product electronicProduct = new Electronics();

            Console.Write("Enter the product Name: ");
            string name = Console.ReadLine();
            electronicProduct.Name = name;

            Console.Write("Enter the product Price: ");
            double price = Convert.ToDouble(Console.ReadLine());
            electronicProduct.Price = price;

            double finalPrice = electronicProduct.CalculateDiscount();

            Console.WriteLine($"The final price of the electronic product {electronicProduct.Name} after discount is: {finalPrice}");
        }
    }
}

```

```

        Product clothingProduct = new Clothing();
        Console.Write("Enter the Clothing type: ");
        string type = Console.ReadLine();

        Console.Write("Enter the product Price: ");
        double clothingPrice = Convert.ToDouble(Console.ReadLine());
        clothingProduct.Price = clothingPrice;

        double finalClothingPrice = clothingProduct.CalculateDiscount();

        Console.WriteLine("The final price of the clothing product {0} after discount is:
{1}", type, finalClothingPrice);
    }
}

```

OUTPUT:

```

Enter the product Name: laptop
Enter the product Price: 30000
The final price of the electronic product laptop after discount is: 28500
Enter the Clothing type: Jeans
Enter the product Price: 1000
The final price of the clothing product Jeans after discount is: 850

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-17\HandsOnDAY18\HandsOnDAY18\bin\
Debug\HandsOnDAY18.exe (process 27672) exited with code 0 (0x0).
Press any key to close this window . . .|

```

Level-2 Problem 5: Vehicle Rental System

Scenario:

A vehicle rental company wants a system where different vehicle types calculate rental charges differently.

Requirements:

1. Create a base class Vehicle with properties Brand and RentalRatePerDay.
2. Create derived classes Car and Bike.
3. Override CalculateRental(int days) method.
4. Car adds insurance charge of 500 per rental.
5. Bike offers 5% discount on total rental.

Technical Constraints:

- Use encapsulation with proper access modifiers.
- Apply runtime polymorphism.
- Validate number of rental days.

Expectations:

- Use base class reference to call overridden methods.
- Implement clean class hierarchy.
- Display final rental cost.

Learning Outcome:

- Master inheritance and polymorphism.
- Implement real-world OOP scenarios.
- Improve object-oriented design skills.

Sample Input:

Car RentalRatePerDay = 2000, Days = 3

Sample Output:

Total Rental = 6500

CODE:

```
using System;

namespace HandsOnDAY18
{
    class Vehicle
    {
        // Private fields
        private string brand;
        private double rentalRatePerDay;

        // Property for Brand
        public string Brand
        {
            get { return brand; }
            set { brand = value; }
        }

        // Property for RentalRatePerDay with validation
        public double RentalRatePerDay
        {
            get { return rentalRatePerDay; }
            set
            {
                if (value < 0)
                {
                    Console.WriteLine("Rental rate cannot be negative");
                }
                else
                {
                    rentalRatePerDay = value;
                }
            }
        }

        // Virtual method
        public virtual double CalculateRental(int days)
        {
            return RentalRatePerDay * days;
        }
    }

    class Car : Vehicle
    {
        public override double CalculateRental(int days)
        {
            if (days <= 0)
            {
                Console.WriteLine("Invalid number of days");
                return 0;
            }

            double total = RentalRatePerDay * days;
            return total + 500;
        }
    }

    class Bike : Vehicle
    {
        public override double CalculateRental(int days)
        {
            if (days <= 0)
            {
```

```

        Console.WriteLine("Invalid number of days");
        return 0;
    }

    double total = RentalRatePerDay * days;
    return total - (total * 0.05);
}

internal class Vehicle_Rental_System
{
    static void Main(string[] args)
    {
        Vehicle car = new Car();
        car.Brand = "Toyota";
        car.RentalRatePerDay = 2000;

        int days = 3;

        double totalRental = car.CalculateRental(days);

        Console.WriteLine("Total Rental = " + totalRental);
    }
}

```

OUTPUT:

```

Total Rental = 6500

C:\Users\keerthi\OneDrive\Desktop\upGrad\DAY-17\HandsOnDAY18\HandsOnD
AY18\bin\Debug\HandsOnDAY18.exe (process 17616) exited with code 0 (0
x0).
Press any key to close this window . . .|

```